

AIR TRACKER

Flight Analytics — Project Documentation

An end-to-end aviation data platform: API ingestion → PostgreSQL warehouse → SQL analytics → interactive Streamlit dashboard.

Domain	Aviation / Data Analytics
Tech Stack	Python · PostgreSQL · AeroDataBox API · Streamlit · Plotly
Author	Jeffrey Gabriel
Document	Workflow · Schema Design · SQL Analytics · Challenges · Insights

1. Project Overview

Air Tracker is a comprehensive flight-analytics solution that ingests live aviation data from the **AeroDataBox API** (via RapidAPI), transforms deeply nested JSON responses into a clean relational schema in **PostgreSQL**, and surfaces operational insight through SQL analytics and an interactive **Streamlit** dashboard. It is built for aviation enthusiasts, analysts, and operations teams who need to understand airport networks, flight patterns, and delay behaviour.

Business Use Cases

- **Airport Exploration** — browse airport details including city, country, continent and timezone.
- **Flight Trend Analysis** — visualise the distribution of flights by status and airline.
- **Operational Insights** — analyse delays and cancellations across selected airports.
- **Decision Support** — data-driven leaderboards (busiest routes, most-delayed airports) for scheduling and resource planning.

Skills Demonstrated

- Data extraction — REST API integration and nested-JSON parsing.
- SQL database management — schema design, keys and constraints.
- Data analysis — 11 analytical SQL queries using JOINS, CASE WHEN, subqueries and aggregation.
- Visualisation — an interactive, themed Streamlit dashboard with Plotly charts.

2. System Architecture & Workflow

The project follows a linear ETL-to-analytics pipeline. Data flows in one direction, from the external API through staging in Python to durable storage in PostgreSQL, and is finally read back by the dashboard at query time.

1 · EXTRACT	2 · TRANSFORM	3 · LOAD	4 · SERVE
AeroDataBox API (RapidAPI) airports · flights aircraft · delays	Python / Pandas flatten nested JSON map fields to columns	PostgreSQL 4 relational tables upserts on conflict	Streamlit + Plotly live SQL queries charts & tables

Data Collection Scope

Fifteen high-traffic airports were selected across the Middle East, Europe, North America, Asia-Pacific and India:

DXB SIN LHR JFK SYD DOH BKK CDG
MAA DEL BOM BLR HYD CCU COK

For each airport the pipeline fetches all departing and arriving flights for the previous day (direction=Both, cancelled flights included). The distinct aircraft registrations seen in the flights table are then used to fetch each aircraft's detail from the aircrafts endpoint, and airport-level delay statistics are retrieved through the statistical delays endpoint using each airport's ICAO code.

Pipeline Stages (air_tracker.ipynb)

Stage	Endpoint	Target Table
Fetch airports	/airports/iata/{code}	airport
Fetch flights (prev. day)	/flights/airports/iata/{code}	flights
Fetch aircraft by registration	/aircrafts/reg/{reg}	aircraft

Fetch delay stats	/airports/icao/{code}/delays	airport_delays
-------------------	------------------------------	----------------

Resilience & Error Handling

- Every API call is wrapped in `try/except` with `raise_for_status()` so HTTP errors are caught rather than crashing the run.
- Airport fetches use a retry loop (up to 3 attempts with back-off) to survive transient RapidAPI rate limits.
- `time.sleep()` throttling between calls keeps the run inside the free-tier request budget.
- Inserts use `ON CONFLICT DO NOTHING` so the pipeline is idempotent and safely re-runnable.

3. Database Schema Design

The relational model normalises the AeroDataBox JSON into four tables. `airport` and `flights` are linked through IATA codes (origin / destination), while `airport_delays` references an airport by its IATA code. Primary keys and UNIQUE constraints enforce data integrity across the schema.

airport

Column	Type	Notes
airport_id	SERIAL	Primary key (auto-increment)
icao_code	TEXT UNIQUE	4-letter ICAO identifier
iata_code	TEXT UNIQUE	3-letter IATA identifier (FK target)
name	TEXT	Full airport name
city	TEXT	Municipality
country	TEXT	Country name
continent	TEXT	Continent name
latitude	REAL	Geographic latitude
longitude	REAL	Geographic longitude
timezone	TEXT	IANA timezone

aircraft

Column	Type	Notes
aircraft_id	SERIAL	Primary key (auto-increment)
registration	TEXT UNIQUE	Tail registration
model	TEXT	Aircraft model
manufacturer	TEXT	Manufacturer name
icao_type_code	TEXT	ICAO type designator
owner	TEXT	Operating airline / owner

flights

Column	Type	Notes
flight_id	TEXT PK	Unique flight identifier
flight_number	TEXT	Public flight number
aircraft_model	TEXT	Aircraft model
aircraft_registration	TEXT	Tail registration (links to aircraft)
origin_iata	TEXT	Departure airport IATA
destination_iata	TEXT	Arrival airport IATA
scheduled_departure	TEXT	Scheduled local departure
actual_departure	TEXT	Actual local departure
scheduled_arrival	TEXT	Scheduled local arrival
actual_arrival	TEXT	Actual local arrival
status	TEXT	Arrived / Departed / Delayed / Canceled ...
airline_code	TEXT	Operating airline IATA

airport_delays

Column	Type	Notes
delay_id	SERIAL	Primary key (auto-increment)
airport_iata	TEXT	Airport IATA code
delay_date	TEXT	Reporting window
total_flights	INTEGER	Total flights in window
delayed_flights	INTEGER	Qualified delayed flights
avg_delay_min	INTEGER	Average delay (minutes)
median_delay_min	INTEGER	Median delay (minutes)
canceled_flights	INTEGER	Cancelled flights

Schema DDL

The PostgreSQL data-definition statements used to create the warehouse:

```
CREATE TABLE airport (  
    airport_id SERIAL PRIMARY KEY,  
    icao_code TEXT UNIQUE,  
    iata_code TEXT UNIQUE,  
    name TEXT,  
    city TEXT,  
    country TEXT,  
    continent TEXT,  
    latitude REAL,  
    longitude REAL,  
    timezone TEXT  
);  
  
CREATE TABLE aircraft (  
    aircraft_id SERIAL PRIMARY KEY,  
    registration TEXT UNIQUE,  
    model TEXT,  
    manufacturer TEXT,  
    icao_type_code TEXT,  
    owner TEXT  
);  
  
CREATE TABLE flights (  
    flight_id TEXT PRIMARY KEY,  
    flight_number TEXT,  
    aircraft_model TEXT,  
    aircraft_registration TEXT,  
    origin_iata TEXT,  
    destination_iata TEXT,  
    scheduled_departure TEXT,  
    actual_departure TEXT,  
    scheduled_arrival TEXT,  
    actual_arrival TEXT,  
    status TEXT,  
    airline_code TEXT  
);  
  
CREATE TABLE airport_delays (  
    delay_id SERIAL PRIMARY KEY,  
    airport_iata TEXT,  
    delay_date TEXT,  
    total_flights INTEGER,  
    delayed_flights INTEGER,  
    avg_delay_min INTEGER,  
    median_delay_min INTEGER,  
    canceled_flights INTEGER  
);
```

4. SQL Analytics

Eleven analytical queries (full text in `queries.sql`) derive operational insight from the warehouse. They exercise aggregation, HAVING filters, multi-table JOINS, correlated subqueries and CASE WHEN classification.

#	Insight	SQL Techniques
1	Total flights per aircraft model	GROUP BY, COUNT
2	Aircraft assigned to more than 5 flights	GROUP BY, HAVING
3	Airports with more than 5 outbound flights	JOIN, HAVING
4	Top 3 destination airports by arrivals	JOIN, ORDER BY, LIMIT
5	Domestic vs International classification	CASE WHEN, self-JOIN on country
6	5 most recent arrivals at DEL	JOIN, filter, ORDER BY, LIMIT
7	Airports never used as a destination	NOT IN subquery
8	Flight count by status per airline	Conditional COUNT(CASE WHEN)
9	All cancelled flights with both airports	LEFT JOIN, ORDER BY
10	City pairs with >2 aircraft models	COUNT(DISTINCT), HAVING
11	% delayed flights per destination airport	Ratio of conditional counts

Sample Query — Delay % per Destination (O11)

```
SELECT a.name AS destination_airport,  
       COUNT(*) AS total_arrivals,  
       COUNT(CASE WHEN f.status='Delayed' THEN 1 END) AS delayed_flights,  
       ROUND(COUNT(CASE WHEN f.status='Delayed' THEN 1 END)  
             * 100.0 / COUNT(*), 2) AS delay_percentage  
FROM flights f  
JOIN airport a ON f.destination_iata = a.iata_code  
GROUP BY a.name  
HAVING COUNT(*) > 0  
ORDER BY delay_percentage DESC;
```

5. Streamlit Dashboard

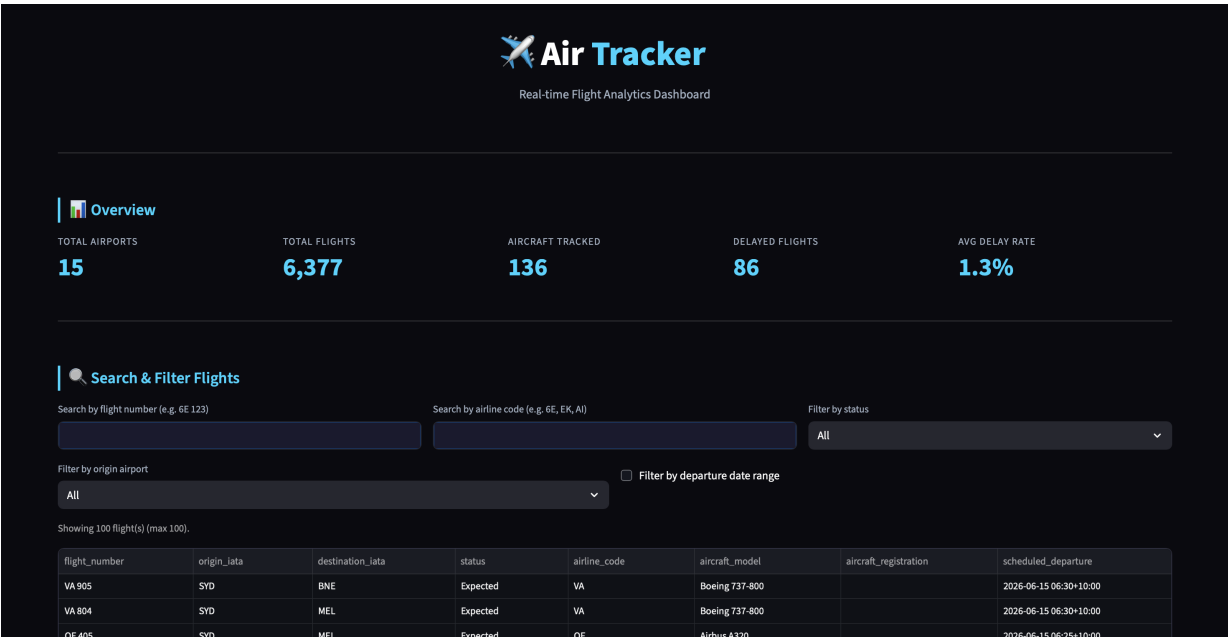
The dashboard (app.py) connects directly to PostgreSQL with psycopg2. All queries are parameterised (no string interpolation of user input) and cached for five minutes with `st.cache_data` for responsiveness. A custom dark, aviation-themed CSS layer styles metric cards, tables and Plotly charts.

Feature Set

- **Homepage dashboard** — KPI cards for total airports, total flights, aircraft tracked, delayed flights and the average delay rate across airports.
- **Search & filter flights** — search by flight number or airline code; filter by status, origin airport and departure date range.
- **Airport details viewer** — select a single airport to see its city, country, IATA/ICAO, timezone and all linked (inbound + outbound) flights.
- **Delay analysis** — bar chart of delay percentage by airport plus a pie chart of flight-status distribution.
- **Route leaderboards** — side-by-side tables of the busiest origin–destination routes and the most-delayed airports.

Dashboard Screenshots

Note: screenshots should be refreshed after running the latest dashboard build.



Homepage dashboard — KPI metric cards.

Search & Filter Flights

Search by flight number (e.g. 6E 123)

Search by airline code (e.g. 6E, EK, AI)

Filter by status

All

Filter by origin airport

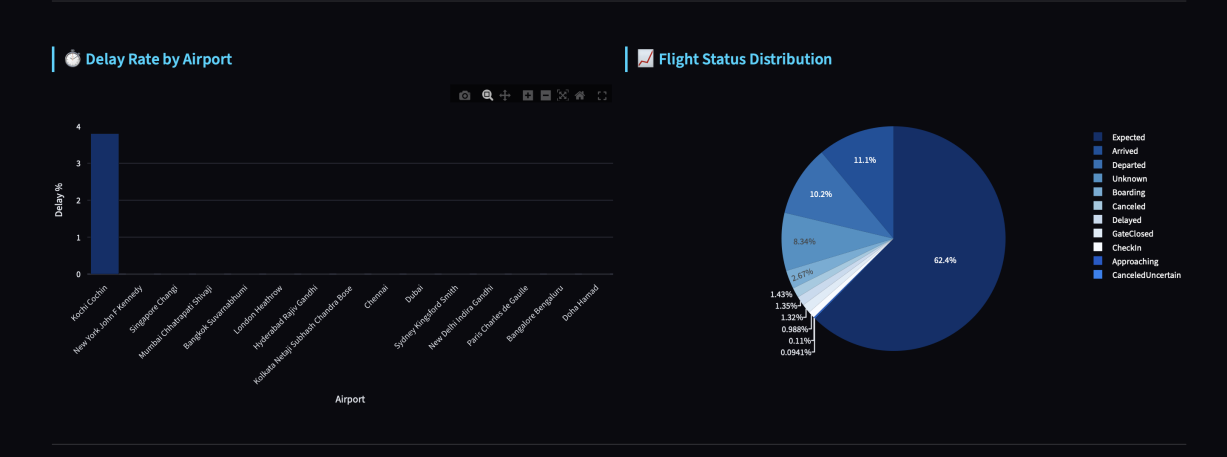
All

Filter by departure date range

Showing 100 flight(s) (max 100).

flight_number	origin_iata	destination_iata	status	airline_code	aircraft_model	aircraft_registration	scheduled_departure
VA 905	SYD	BNE	Expected	VA	Boeing 737-800		2026-06-15 06:30+10:00
VA 804	SYD	MEL	Expected	VA	Boeing 737-800		2026-06-15 06:30+10:00
QF 405	SYD	MEL	Expected	QF	Airbus A320		2026-06-15 06:25+10:00
QF 731	SYD	ADL	Expected	QF	Boeing 737-800		2026-06-15 06:25+10:00
QF 502	SYD	BNE	Expected	QF	Boeing 737-800	VH-OGF	2026-06-15 06:25+10:00
FJ 914	SYD	NAN	Expected	FJ	Boeing 737-800	DQ-FAF	2026-06-15 06:15+10:00
JQ 39	SYD	DPS	Expected	JQ	Airbus A320	VH-OFS	2026-06-15 06:15+10:00
VA 800	SYD	MEL	Expected	VA	Boeing 737-800		2026-06-15 06:15+10:00
VA 549	SYD	PER	Expected	VA	Boeing 737-800		2026-06-15 06:15+10:00
JQ 950	SYD	CNS	Expected	JQ	Airbus A320		2026-06-15 06:10+10:00

Search & filter flights.



Delay analysis — delay rate by airport and status distribution.

Airport Details Viewer

Select an airport

Bangalore Bengaluru

CITY

COUNTRY

IATA / ICAO

TIMEZONE

Bangalore

India

BLR / VOBL

Asia/Kolkata

Linked flights at BLR: 100 (max 100 shown)

flight_number	origin_iata	destination_iata	status	airline_code	scheduled_departure
OG 160	BLR		Approaching	OG	
6E 7177	BLR		Unknown	6E	
6E 509	BLR		Expected	6E	
6E 7736	BLR		Expected	6E	
SG 559	BLR		Expected	SG	
SG 572	BLR		Unknown	SG	
AI 2812	BOM	BLR	Expected	AI	2026-06-15 02:05+05:30
6E 1213	BLR	BKK	Expected	6E	2026-06-15 01:40+05:30
AF 191	BLR	CDG	Expected	AF	2026-06-15 01:25+05:30
6E 1419	BLR	AUH	Expected	6E	2026-06-15 01:20+05:30

Airport details viewer.

🏆 Busiest Routes			🛫 Most Delayed Airports			
origin_iata	destination_iata	flight_count	name	total_flights	delayed_flights	delay_pct
DEL	BOM	31	Kochi Cochin	26	1	3.8
SYD	MEL	22	New York John F Kennedy	23	0	0
DEL	BLR	21	Singapore Changi	20	0	0
JFK	LAX	18	Mumbai Chhatrapati Shivaji	69	0	0
LHR	JFK	18	Bangkok Suvarnabhumi	23	0	0
SIN	KUL	16	London Heathrow	14	0	0
BOM	BLR	16	Hyderabad Rajiv Gandhi	45	0	0
DEL	PNQ	15	Kolkata Netaji Subhash Chandra Bose	40	0	0
CDG	NCE	15	Chennai	10	0	0
DEL	HYD	15	Dubai	38	0	0

Route leaderboards — busiest routes and most-delayed airports.

6. Challenges & Resolutions

Challenge	Problem	Resolution
RapidAPI rate limits	Free-tier request caps caused intermittent failures.	Added retry loops with back-off and sleep throttling between every call.
Deeply nested JSON	Flight records nest airport, aircraft, airline and time objects several levels deep.	Used chained .get() with defaults to flatten safely without KeyErrors.
Inconsistent airline identifiers	The aircraft endpoint rejected some airline name formats.	Standardised on IATA codes for a curated list of top airlines.
ICAO vs IATA mismatch	The delays endpoint requires ICAO, but the schema keys on IATA.	Maintained an explicit IATA→ICAO map to bridge the two endpoints.
Idempotent re-runs	Re-running the notebook risked duplicate rows.	Applied ON CONFLICT DO NOTHING on the unique keys of each table.

7. Key Insights

- Indian metros (DEL, BOM, MAA, BLR) dominate the busiest-route leaderboard within the selected sample.
- Delay percentage varies sharply by destination airport, making it a stronger operational signal than a raw delay count.
- Conditional aggregation (COUNT(CASE WHEN ...)) cleanly pivots flight status into per-airline columns without multiple passes.
- Most routes are served by a single aircraft model; multi-model city pairs flag high-frequency trunk routes.

8. Evaluation Criteria Mapping

Metric	How It Is Met
Data Extraction Accuracy	Retry + error handling around every AeroDataBox endpoint.
SQL Database Design	Four normalised tables, UNIQUE constraints, IATA-based relationships.
Query Efficiency	Aggregation and filtering pushed into SQL; JOINS on keyed IATA columns.
Dashboard Functionality	Six interactive sections with filters, charts and leaderboards.
Project Completeness	Full path: API → PostgreSQL → SQL → Streamlit.
Error Handling	try/except, raise_for_status, rate-limit back-off, ON CONFLICT upserts.
Documentation	This report plus README and queries.sql in the repository.

9. Installation & Setup Guide

Follow these steps to run Air Tracker from a clean machine. The pipeline must be run once to populate the database before the dashboard will display any data.

Prerequisites

- **Python 3.9+** with pip.
- **PostgreSQL 13+** running locally (or a reachable instance).
- A free **RapidAPI** account with a subscription to the **AeroDataBox** API (provides the API key).

Step 1 — Clone & install dependencies

```
git clone https://github.com/Jeffrey-29/air-tracker-flight-analytics.git
cd air-tracker-flight-analytics
pip3 install -r requirements.txt
```

Step 2 — Create the database & schema

Create the PostgreSQL database, then create the four tables using the DDL from Section 3:

```
# create the database
createdb air_tracker          # or: CREATE DATABASE air_tracker; in psql

# load the schema (run the CREATE TABLE statements from Section 3)
psql -d air_tracker -f schema.sql
```

Step 3 — Configure environment variables

Create a `.env` file in the project root with your database credentials and AeroDataBox API key:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=air_tracker
DB_USER=postgres
DB_PASSWORD=your_password

API_KEY=your_rapidapi_key
```

Step 4 — Populate the data

Run the ETL notebook to fetch data from AeroDataBox and load it into PostgreSQL. Execute the cells in order — airports first, then flights, aircraft and delays:

```
jupyter notebook air_tracker.ipynb
# run all cells top to bottom
```

Step 5 — Launch the dashboard

```
streamlit run app.py
# opens at http://localhost:8501
```

Note: The AeroDataBox free tier is rate-limited. If a fetch fails, the notebook retries automatically — simply re-run the cell. Inserts use `ON CONFLICT DO NOTHING`, so re-running is safe and never duplicates rows.

10. Repository Structure

```
air-tracker/
  app.py          # Streamlit dashboard
  air_tracker.ipynb # ETL pipeline (extract -> load)
```

```
queries.sql      # 11 analytical SQL queries
schema.sql       # CREATE TABLE statements (Section 3)
requirements.txt  # Python dependencies
README.md        # Setup & overview
screenshots/     # Dashboard images
```